

P3233R0

Issues with P2786 ("Trivial Relocatability For C++26")

Published Proposal, 2024-04-13

Author:

[Giuseppe D'Angelo](#)

Audience:

EWG, LEWG, SG12

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

This paper highlights some issues with the currently approved design for trivial relocation in the C++ language.

Table of Contents

- 1 Changelog**
- 2 Introduction**
 - 2.1 What this paper is **not**
- 3 Why have TR in the language at all?**
- 4 The status quo**
- 5 What optimizations are possible in the current model?**
- 6 The missing optimizations**
 - 6.1 Why are TR optimizations for move assignments not allowed?
 - 6.2 Does anyone even use this specific optimization for move assignments?

- 6.3 Optimizing swaps
- 6.4 Can we "just" reformulate `vector::insert` / `erase` / etc. not to do assignments but constructions/destructions?
- 6.5 Instead of adding another language facility for this optimization, can we use reflection instead?
- 6.6 Can the current model of TR be extended to cover the move-assignment optimizations, in the language, *at a later stage*?

7 Lack of library API

- 7.1 Do we know we got the design right?
- 7.2 Leaving the TR status of Standard Library datatypes as QoI

8 What even is relocation?

- 8.1 Is TR its own primitive operation on a type?
- 8.2 Why allowing for trivially copyable types that aren't TR?
- 8.3 Unclear behaviors for polymorphic classes

9 Enforcement model

- 9.1 What is the cost/benefit ratio of enforcement?
- 9.2 Automatic TR is still not entirely safe

10 P1144's trivial relocation model

- 10.1 Overall differences between P2786 and P1144
- 10.2 P1144's TR semantics
 - 10.2.1 Trivially copyable always implies automatically trivially relocatable
 - 10.2.2 TR is an intrinsic quality
 - 10.2.3 Trivially copyable types are a subset of trivially relocatable
- 10.3 Which optimizations are possible in P1144's model?
- 10.4 Which optimizations are **not possible** in P1144's model?

11 Summary

12 A possible way forward

- 12.1 Disclaimer
- 12.2 Proposed action points

13 Acknowledgements

References

Informative References

§ 1. Changelog

- R0
 - First submission

§ 2. Introduction

In the Tokyo 2024 meeting [\[P2786R4\]](#) ("Trivial Relocatability For C++26") was adopted by EWG. **We believe that the design for trivial relocation of that paper is too restrictive**, and blocks a significant amount of optimizations. Not only this means that the Standard Library won't get as many benefits as it could, but also third-party libraries (such as Qt, Folly, BSL, etc.) that define their own trivial relocation model may not be able to fully adopt the Standard's.

We also fear that the "vocabulary" for trivial relocation standardized by [\[P2786R4\]](#) risks limiting future extensions.

On the other hand, [\[P1144R10\]](#) ("std::is_trivially_relocatable")'s design for trivial relocation allows for some **different** optimizations. These optimizations more closely match existing practices, but do not necessarily include the ones that [\[P2786R4\]](#) allows for.

In other words: **there is a split between the two proposals w.r.t. what can be effectively optimized** (and how). We believe that this is a subtle but very important difference that the "comparison paper" [\[P2814R0\]](#) might not have emphasized enough; this aspect got possibly entangled with the long listing of minutiae about syntactic differences.

Instead, we believe that a design that satisfies both [\[P2786R4\]](#) and [\[P1144R10\]](#) can and should be found.

Editorial note: in the rest of this paper, for the sake of brevity, we are going to use "TR" instead of "trivial relocation" / "trivially relocatable" / "trivial relocatability" and similar. We hope that this will not cause any confusion for the reader.

§ 2.1. What this paper is **not**

This paper tries to focus on some high-level issues with the two competing proposals for trivial relocation in C++. We will deliberately try to avoid any discussions regarding the low-level mechanics, such as "should TR be expressed via a keyword or via an attribute", "what is precisely the API to (trivially) relocate an object", "should we have a type trait or a concept" and so on. While these practical aspects are important, we fear that these discussions risk distracting from the broader picture.

This paper is not a blanket endorsement of [\[P1144R10\]](#)'s trivial relocation design, as that one also has some possible shortcomings that may leave some users unsatisfied (which is likely why [\[P2786R4\]](#) was introduced). However, if we were to choose only one between the two proposals as they exist today, we would prefer [\[P1144R10\]](#).

Finally, this paper is also not an "oven-ready counter-proposal" to either [\[P2786R4\]](#) or [\[P1144R10\]](#). We do not have a complete solution for many of the problems that we are going to highlight, only ideas and sketches for a way forward.

§ 3. Why have TR in the language at all?

The main driver behind the TR proposals (and the existing usage of TR in third-party libraries) is to unlock a family of optimization techniques. These techniques have a common theme: turn certain combinations of move operations and/or destruction operations into byte manipulations. These manipulations are already available for types that are trivially copyable, but are not allowed for many other useful types (such as `std::unique_ptr<T>`).

The main reasons behind adding a proper definition of TR to the language are:

1. **to precisely define what are the characteristics of a type so that it's eligible for TR** (that is, it can be manipulated at the byte level instead of requiring calls to move operations, destructors, etc.);
2. **to give proper names and semantics to these byte level operations**, especially concerning the lifetime of the objects involved. Without support in the language, existing libraries that use these TR optimizations always tread on *"Undefined Behavior That Works In Practice"*.

Indeed, this kind of low-level object manipulations is an area that (historically) has always been full of such UBTWIP. Thankfully, more and more facilities are being added to reduce this source of UB. In this sense, language support for TR is an important step in this direction, and we strongly welcome it.

3. **to allow type properties to naturally propagate and compose when defining new types**. This is completely in line with how other type properties already work and propagate: copyability, trivial copyability, and so on.

For instance, "Rule Of Zero" types should automatically be TR if their subobjects can be TR; this automatic propagation is only possible via a language mechanism, not via a library one (especially lacking reflection):

```
// Given this:
class A TRIVIAALLY_RELOCATABLE_TAG { ~~~ };
static_assert(std::is_trivially_relocatable_v<A>);
```

```
// We want this to hold automatically:  
class B { A a; };  
static_assert(std::is_trivially_relocatable_v<B>);
```

§ 4. The status quo

The following is a summary of the currently approved language design for TR in [\[P2786R4\]](#).

- C++26 types will have a new property/category called "trivially relocatable", comprising scalars, TR classes, arrays of TR types and cv-qualified versions of TR types.
- A class can be manually marked as TR (or not) by using a new contextual keyword (`trivially_relocatable`, with an optional boolean argument to be able to express conditional TR).
- In alternative, an *unmarked* class is automatically TR if:
 - all of its subobjects are of TR type or of reference type; and
 - it has no virtual bases; and
 - it has a non-deleted non-user-provided destructor; and
 - it is move constructible via a non-deleted non-user-provided constructor.
- In case a class is manually marked as TR and has either a non-TR subobject or a virtual base, then the program is ill-formed. In other words, the language actively checks that a type marked as TR also supports TR for all of its subobjects.

§ 5. What optimizations are possible in the current model?

Given an object `A` of a TR type `T`, one may turn the sequence of these two operations:

1. move construction of an existing object `A` into a new object `B`;
2. followed by the destruction of `A`;

into a mere byte copy (e.g. `memcpy` or `memmove`) from the storage of `A` into the storage where to build `B`. In particular, *no calls to the move constructor or to the destructor take place*. The storage for `A` can then be reused or freed.

Due to the aforementioned lifetime concerns, code that wants to use this optimization can't *literally* apply `memcpy` without running into UB, because a TR type is not necessarily trivially copyable. Instead, a new facility is provided in order to perform TR, with the idea that the facility performs (the equivalent of) a `memcpy`, as well as correctly starts and ends the lifetimes in the abstract machine for the involved objects.

The exact shape of the facility (keyword, library function, ...) is not relevant for the purpose of the present paper.

In pseudocode:

```
// Given:
T *ptrA = ~~~;          // alive
T *storageForB = ~~~; // uninitialized

// Without trivial relocation:
new (storageForB) T(std::move(*ptrA));
ptrA->~T();

// With trivial relocation: call a facility that performs the equivalent of
std::memcpy(storageForB, ptrA, sizeof(T));
end_lifetime(ptrA);
start_lifetime(storageForB);
```

The "textbook example" of a non-trivially copyable type that has suitable characteristics for this kind of optimization is `std::unique_ptr<int>`.

If we apply this logic in bulk, to multiple contiguous objects, the byte operations can be coalesced into one big `memcpy/memmove` call. This has a significant performance benefit in terms of execution speed, as well as interesting secondary effects (fewer template instantiations, reducing build times and code bloat); both [\[P2786R4\]](#) and [\[P1144R10\]](#) provide benchmarks.

The "textbook example" where this optimization is usable is during the reallocation of a `std::vector`, where objects are move-constructed from the old storage into some newly allocated storage, and then the old objects are destroyed and the old storage is released.

This allows to reallocate a vector-like container by simply calling `std::realloc` for a further performance benefit. `std::vector`, being allocator-aware, cannot use `realloc` yet because such an API is missing from allocators.



It's important to underline that the current requirements for TR types are meant to support this optimization model. In particular:

- a type isn't automatically TR if it has user-defined move constructors or destructors. If we cannot reason about what those operations do, we cannot claim in general that they're equivalent to a `memcpy`;

- we must exclude classes with virtual bases, because virtual bases may be implemented via self-referential pointers (that is, pointers that point into the object itself), which therefore makes objects of these classes not relocatable via a mere `memcpy` (the pointers need to be adjusted);
 - on the other hand, classes with virtual functions can be TR in the current model, because the virtual pointers just point to a static piece of information (the virtual table of the type) which is stored *outside* of the object itself (on all common ABIs);
- the **presence of assignment operators** is not considered at all, because in this model we don't optimize assignments, only move-construction and destruction.

§ 6. The missing optimizations

There is another TR-related optimization which is not automatically allowed by the currently model: **using TR in order to optimize move *assignments* as well as move constructions / destructions.**

Consider for instance a `std::vector<X>` which is not at full capacity.

An insertion in the middle of the vector requires "shifting" all the elements at and after the insertion position (the "tail"), in order to make room for the new element to insert. This is realized via a move construction (the last element of the vector gets move-constructed into the uninitialized storage, into the vector's extra capacity), followed by a series of move assignments for the rest of the tail of the vector.

Finally, we can copy or move-assign the element to insert into its final position.

If `X` satisfies certain type requirements, instead of doing this sequence of a move construction and assignments, we could just `memmove` the contents of the vector by one position "to the right" in order to make room for the new element.

In pseudocode:

```
template <typename T>
vector<T>::insert(size_t position, T &&t)
{
    // ... separately deal with "full capacity" ...
    // ... separately deal with append at the end ...

    assert(m_size < m_capacity);

    if constexpr ( /* does T support this optimization? */ ) {
        // bitwise move the tail one position forward, to create room
        std::memmove(m_begin + position + 1,
                    m_begin + position,
                    (m_size - position) * sizeof(T));
```

```

        // move construct the element to be inserted in the "window" so created
        new (m_begin + position) T(std::move(t));
    } else {
        // move construct in the extra capacity
        new (m_begin + m_size) T(std::move(m_begin[m_size - 1]));
        // move assign the tail one position forward
        std::move_backward(m_begin + position,
                           m_begin + m_size - 1,
                           m_begin + m_size);
        // move assign the element to be inserted in the moved-from "window"
        m_begin[position] = std::move(t);
    }

    // update bookkeeping
}

```

The same kind of reasoning can be applied for *erasing* an element in the middle of a vector. Normally, erasing involves a number of move-assignments "to the left", overwriting the element that needs to be destroyed, and then destroying the moved-from last element of the vector.

Again, given a suitable type `X`, `std::vector<X>::erase` could instead use this strategy:

- destroy the element;
- `memmove` the tail one position to the left.



These optimizations for insert/erase can and are employed *today*, for instance, if `X` is a trivially copyable type. Once more `std::unique_ptr<T>` is the textbook example of a non-trivially copyable type for which these optimizations are in principle possible.

§ 6.1. Why are TR optimizations for move assignments not allowed?

In the currently adopted model ([P2786R4]) one can have types that are TR, yet have user-defined assignment operators. Consider a type with reference semantics like:

```

struct IRef {
    int &ref;
    IRef &operator=(const IRef &other) { ref = other.ref; return *this; }
}

```



```
};
```

This type is automatically trivially relocatable in the currently adopted model. It has:

- no non-TR subobjects;
- no virtual base classes;
- no user-defined or deleted move constructors or destructors,

and therefore it satisfies all the criteria for being implicitly TR.

This is OK, in the sense that it is perfectly fine to use `memcpy` in order to reallocate a `std::vector<IRef>` (TR instead of move construction + destruction, as discussed above).

This means that an operation such as erasing an element from such a vector **must** be implemented via a series of move-assignments and a destruction, because one can see the side-effects of such assignments.

Turning the *assignments* into manipulation of bytes is not allowed for something like `IRef`.

Examples of types with `IRef` semantics are for instance `std::tuple<int &>`, or `std::pmr` types like `std::pmr::string`.



The conclusion is that **the currently adopted model for TR does not distinguish between `IRef` (TR for move construction only) and something like `std::unique_ptr<T>` (TR for move construction and move assignment).**

Ultimately, it boils down to the fact that there are types for which move assignment is "equivalent" to destruction+move construction, and types for which this isn't, and the current TR model does not distinguish between the two.

This last remark is important, because it underlines the fact that the optimization for move assignments is a generalization of the currently adopted TR model. If a move assignment for a TR type `T` is equivalent to destruction of the target and move construction from the source, it also means that we can turn this sequence:

```
T *target = ~~~; // alive
T *source = ~~~; // alive

*target = std::move(*source);
source->~T();
```

into

```
target->~T();
trivially_relocate(source, target); // as per the currently adopted model
```

which is what we want to exploit for instance in `vector::erase`.

Since there is no distinction in the language (`IRef` and `std::unique_ptr<int>` are both TR and non-trivially move assignable), it follows that **in the currently adopted model we cannot apply any TR optimization for move assignments**; `erase` and `insert` into vectors *cannot* be optimized as shown earlier.

It is important to note that [P1144R10]'s TR model we would instead have optimization, because that model checks for the presence of user-defined assignment operators. We will discuss this later in the paper.

§ 6.2. Does anyone even use this specific optimization for move assignments?

Many third party libraries (Qt, Folly, Abseil, BSL) do use it; [P1144R10] has a thorough survey in §2.1.

The reason for the "popularity" of this optimization for move assignments is that it is in principle available for a *huge* number of vocabulary types: `std::unique_ptr`, `std::shared_ptr`, most container types, string types (but not necessarily `std::string`, which may not be TR at all), `std::optional` and `std::variant` (depending on the types they hold), and so on.

Custom allocators/deleters of course play a role in this, in the sense that they may do have reference semantics; but we believe that it is unfair to "globally" impede this optimization for everyone. It should just get forbidden for such specific deleters or allocators.

In conclusion, it is our opinion that **it is extremely suboptimal to miss this optimization opportunity**, given the widespread precedent for it.

§ 6.3. Optimizing swaps

Allowing the optimization of move assignments is also a key building block into making trivially swappable types; and, consequently, **optimize swap-based algorithms via TR**. All these optimization opportunities are unavailable in the currently adopted model.

§ 6.4. Can we "just" reformulate `vector::insert` / `erase` / etc. not to do assignments but constructions/destructions?

While certainly possible in terms of abstract design, such a change would constitute an API break: several of those operations either specify to use assignments, or they de-facto do them today and a change would break existing code (Hyrum's law).

Given the longevity of these APIs and their centrality, we therefore do not believe that such a break would be even remotely acceptable.

[P2959R0] ("Container Relocation") discusses a library-based approach to allow TR for move assignments. The key building block is a new type trait (called `container_replace_with_assignment` in the paper). By default, this trait would keep the existing semantics (use move assignments); however, types will be able to opt into the trait, and therefore enable `erase`, `insert` and similar operations to use TR.

We strongly reject a library-based approach for this use case.

As we discussed in the introduction, a *language* approach is necessary for this property to naturally propagate and compose:

```
// Given:
struct A { ~~~ };
static_assert(std::is_trivially_relocatable_v<A>);

// Mark the type
template <>
inline constexpr bool std::container_replace_with_assignment<A> = false;

// Then:
struct B { A a; };

// Automatically true, because of language rules:
```

```
static_assert(std::is_trivially_relocatable_v<B>);

// FAIL! A library trait isn't propagated.
static_assert(!std::container_replace_with_assignment<B>);
```

Second, as previously noted, the ability of optimizing move assignments via TR is not merely an enabler for containers; it is also a building block for trivial swaps and swap-based algorithms. **The `container_replace_with_assignment` trait is misnamed, and does not bring the necessary attention to the wanted semantics of a type.**

Third, **it clashes (and/or has an unknown relationship) with trivially copyable types**, for which containers and algorithms can already use `memmove` in place of move assignments and construction/destruction.

§ 6.5. Instead of adding another language facility for this optimization, can we use reflection instead?

We are unable to give an accurate answer to this question, as reflection for C++ is still a pending feature.

In principle, it certainly sounds possible to use reflection to create a trait that automatically determines if a type supports TR for move assignments.

We would still need a trait in the standard library, so that classes with user-defined assignment operations can opt-in and declare that they are optimizable (similar to the `container_replace_with_assignment` trait proposed by [\[P2959R0\]](#)).

Therefore, the detection should check if the trait has been opted-in; otherwise, it should check that the type is TR, it has no user-declared assignment operators, and it has no virtual functions (in other words, it should use reflection to do what [\[P1144R10\]](#) does through the language).

In practice, **we reject a reflection-based approach:**

1. it is questionable whether this approach has any advantage at all. Instead of two type properties in the language, we would need a type property and a customizable type trait in the library;
2. we would need a very specific feature from reflection, namely, we need the ability to check whether a class has *user-provided assignment operators*. The current reflection proposal ([\[P2996R2\]](#)) lacks such a query;
3. we believe that these optimizations are very important to have, and therefore we do not want to "tie" their availability to the progress of the reflection proposals.

§ 6.6. Can the current model of TR be extended to cover the move-assignment optimizations, in the language, *at a later stage*?

As discussed above, the currently model is a strict subset of the one where TR is allowed for move assignments. It would therefore be perfectly fine to *relax the model* or *introduce another model* (e.g. another keyword/trait) at a later time.

We are however concerned with the fact that the *current* proposal is reserving the "trivial relocatable" name/vocabulary to indicate a *subset* of the possible use cases for TR. If anything, it should be using a more restrictive name -- leaving the more generic "trivial relocation" one for the wider, later proposals.

As shown above, **existing practice in libraries uses "trivial relocation" to include move assignments.** It is confusing to introduce the same terminology into core language, but with a different meaning.

§ 7. Lack of library API

[\[P2786R4\]](#) also features a minimal library API (consisting of a type trait and a trivial relocation function). Other library extensions are discussed in the aforementioned [\[P2959R0\]](#) ("Container Relocation") and [\[P2967R0\]](#) ("Relocation Is A Library Interface").

§ 7.1. Do we know we got the design right?

Trivial relocation is a feature that first and foremost is an enabler for some optimizations in the Standard Library (and user code). The library additions should have been thoroughly analyzed in order to validate the language changes.

On the other hand, [\[P1144R10\]](#)'s design has already widespread precedent and implementation experience.

§ 7.2. Leaving the TR status of Standard Library datatypes as QoI

We are extremely concerned with the fact that in the adopted design explicit library support is required in order to consume and create TR types. That is, a type that needs to be explicitly marked as TR (for instance, a type that defines its Rule Of Five special member functions) is *required* to be composed of TR subobjects; otherwise, the program is ill-formed.

Since a lack of the TR keyword will affect program well-formedness, we strongly believe that this is not an issue that can be left as QoI in the Standard Library, as it would make programs inherently non-portable:

```
// This *completely reasonable* code may or may not compile:

struct S trivially_relocatable
{
    S();
    S(S &&);
    ~S();

    std::shared_ptr<int> ptr;
};
```

To make the above portable, one could add a boolean condition in the argument of the `trivially_relocatable` keyword (checking all the subobjects), but this is extremely vexing and error-prone.

§ 8. What even is relocation?

The currently adopted model introduces the "trivial relocatable" type property **without also formally defining what "relocatable" or "relocation" means**.

This is a striking difference with all the other fundamental type properties, for which there's a "trivial-less" definition, and the trivial version means "this can be done, and can be done without running specialized code" (for instance, "copy constructible" and "trivially copy constructible"). (The only exception to this pattern would be "trivially copyable", but this is an "umbrella" property defined in terms of other properties, which are required to be trivial.)

In particular: **the adopted model does *not* state anywhere that a relocation is a move construction followed by a destruction of the source object**, and therefore a trivial relocation is meant to achieve the same effects (but since it's trivial, it can be done by using `memcpy` plus lifetime magic).

Without a proper definition of "relocatable", it is hard to reason about what it means in terms of type design, semantics, and its interactions with copy and move operations. **This is at odds with existing practice**, where a definition of relocation *is* provided and its relation with the other elementary class operations is clear. (From this point of view, [\[P1144R10\]](#) defines relocation.)

It is also at odds with the proposed addition of higher-level algorithms like `std::uninitialized_relocate`, which will trivially relocate TR types and "do something else" for non-TR types, once more raising questions on the semantic overlaps.

§ 8.1. Is TR its own primitive operation on a type?

The currently adopted model allows for types that are TR but are for instance not movable (or not publicly movable):

```
struct S trivially_relocatable
{
    S();
    S(const S &) = delete;
    S(S &&) = delete;
    ~S();
};
```

It is very unclear if these types useful in practice — that is, if TR truly constitutes a new, different primitive operation; or if instead such types are "abominations".

If it's the latter, should the compiler check that a type marked as TR is also movable and destructible (i.e. supports some form of non-trivial "relocation")?

For instance, a different but related example of "abomination" is a type that is copyable but not movable:

```
struct CBNM
{
    CBNM();
    CBNM(const CBNM &);
    CBNM(CBNM &&) = delete;
};
```

While formally allowed by the language, such a type has very dubious design and semantics.

§ 8.2. Why allowing for trivially copyable types that aren't TR?

Symmetrically, it is not entirely clear what is the purpose of allowing types to be explicitly marked as non-TR, *even if they are trivially copyable*:

```
// What does this *mean*?
struct TC trivially_relocatable(false)
{
    int x, y;
```

```
};

static_assert(not std::is_trivially_relocatable_v<TC>); // OK
```

This means that, in the adopted model, **trivial relocatable is not a strict superset of trivially copyable**.

This has poor usability for implementors of containers and algorithms, who already optimize trivially copyable types using `memcpy`. In the adopted model it is not allowed to TR a non-TR type even if it's trivially copyable; this will result in vexing "duplicated" code. For instance:

```
template <typename T>
vector<T>::reallocate_impl(size_t new_capacity)
{
    assert(m_size <= new_capacity);
    T *new_storage = allocate(new_capacity);

    // Need to handle TR and TC separately, because it's
    // not allowed to call trivially_relocate on a non-TR type,
    // even if it's TC!
    if constexpr (std::is_trivially_relocatable_v<T>) {
        std::trivially_relocate(m_begin, m_begin + m_size, new_storage);
    } else if constexpr (std::is_trivially_copyable_v<T>) {
        std::memcpy(new_storage, m_begin, m_size * sizeof(T));
    } else if constexpr (std::is_nothrow_move_constructible_v<T>) {
        std::uninitialized_move(m_begin, m_begin + m_size, new_storage);
        std::destroy(m_begin, m_begin + m_size);
    } else {
        // ...
    }

    deallocate(m_begin);
    m_begin = new_storage;
    m_capacity = new_capacity;
}
```

Granted, the adoption of a higher level facility such as `std::uninitialized_relocate` may encapsulate and streamline this logic. The semantic split still presents a burden for reasoning about types, and therefore we would like to see better justification for it.

§ 8.3. Unclear behaviors for polymorphic classes

Polymorphic classes can be implicitly TR in the currently adopted model:

```
struct Base
{
    virtual void f();
    int a;
};

struct Derived : Base
{
    void f() override;
    int b;
};

static_assert(std::is_trivially_relocatable_v<Base>);    // OK
static_assert(std::is_trivially_relocatable_v<Derived>); // OK
```

The use case for making these types TR is to enable the TR optimization for vector reallocation: as explained above, a `std::vector<Base>` can safely use TR in order to reallocate its storage.

Unfortunately, in the proposed model **the TR semantics break down when we try to relocate a single object**. This code is well-formed:

```
Base *source = new Derived;
Base *target = allocate(sizeof(Base));

// What is the behavior here?
std::trivially_relocate(source, source + 1, target);
```

We believe that the TR operation should trigger UB; from a certain point of view, we are destroying a `Derived` object, which does not have a virtual destructor, through a `Base` pointer. However, **it is unclear what is the behavior of this code in the currently approved model**; again the lack of a precise specification of what it is meant by relocation makes it hard to reason about this example.

(It is worth noting that the example's behavior would still be questionable even if the classes involved were not of polymorphic type.)



Suppose that we further modify the example, and add the missing virtual destructor:

```
struct Base
{
    virtual ~Base() = default;
    virtual void f();
    int a;
};

struct Derived : Base
{
    void f() override;
    int b;
};

static_assert(std::is_trivially_relocatable_v<Base>);    // OK
static_assert(std::is_trivially_relocatable_v<Derived>); // OK
```

`Base` (and thus `Derived`) is still implicitly TR, because its destructor is merely user-declared, not user-provided. The code in the previous example:

```
Base *source = new Derived;
Base *target = allocate(sizeof(Base));

// Still not ok!
std::trivially_relocate(source, source + 1, target);
```

is still extremely problematic, because it is copying the byte representation of `Derived` into a new object of type `Base`. In doing so, **the code ends up copying the virtual table pointer from a `Derived` instance into a `Base` object!**

This code will certainly exhibit behavioral issues; for instance, if someone calls `target->f()`, then `Derived::f()` will be called, but `this` points to a `Base` object.

It would not be too far fetched to state that the TR operation *still* causes undefined behavior. **This does not seem to be stated anywhere by the currently adopted model.**

We believe that, at a minimum, the specification for `std::trivially_relocate<T>` should be amended in order to add as a *precondition* that `T` is the most derived type pointed by the arguments. This precondition is satisfied by `std::vector<Base>`, but it not necessarily is when dealing with single objects.

This example also shows that in the currently adopted model it is impossible to avoid UB, even when using types that are automatically TR. This fact somehow undermines the safety claims of embracing an enforcement model for TR.

§ 9. Enforcement model

The currently adopted model adopts enforcement mechanics for classes that are manually marked as `trivially_relocatable(true)`: if the class has virtual bases and/or non-TR subobjects, the program is ill-formed.

This enforcement exists in order to prevent accidental UB: if a class is marked as TR, but one of its subobjects does not actually support trivial relocation, the program is ill-formed rather than exhibiting UB at runtime.

For example:

```
struct SavedFromUB trivially_relocatable
{
    SavedFromUB();
    ~SavedFromUB(); // user-defined dtor => need explicit marking

    std::string s; // possible problem
};
```

On some implementations `std::string` is not actually TR, because it contains a self-referential pointer (the "`begin`" pointer" points into the SSO buffer). Therefore, on those implementations, the program above will be ill-formed, because `std::string` will not be marked TR there.

Virtual base classes may also be implemented via self-referential pointers, and that is why they make a type non TR.

One can also follow a more formal line of reasoning: if TR is a new primitive operation in the language, then all the subobjects of a type must be TR in order for the type to be itself TR.

Of course, a type with all subobjects of TR type and with no virtual base classes can still be accidentally marked as TR even if it is not (again, it may contain self-referential pointers); there is no real way to prevent this from happening and thus avoid UB.

§ 9.1. What is the cost/benefit ratio of enforcement?

An enforcement policy comes with some associated engineering costs.

- Authors of "library types" may split the implementation of a type in a certain number of sub-objects. This is often (but not always) done in order to make the implementation compatible with older C++ standards, which don't offer language facilities to streamline the code.
 - For instance, deleters, comparators, allocators and so on are usually (privately) inherited from in order to exploit the Empty Base Optimization; `[[no_unique_address]]` requires C++20.
 - Containers like `std::variant` or `std::optional` need to enable or disable a number of features (`explicit` construction, copy operations, move operations, ...) depending on the type they hold. Therefore, they cannot use `explicit(bool)`, constraints, and similar "modern" facilities, if they need to be compilable in older C++ standards.

As a concrete example: this is `std::variant` from libc++:

```
template<typename... _Types>
class variant
: private __detail::__variant::__Variant_base<_Types...>,
  private _Enable_default_constructor<
    __detail::__variant::__Traits<_Types...>::__S_default_ctor,
    variant<_Types...>>,
  private _Enable_copy_move<
    __detail::__variant::__Traits<_Types...>::__S_copy_ctor,
    __detail::__variant::__Traits<_Types...>::__S_copy_assign,
    __detail::__variant::__Traits<_Types...>::__S_move_ctor,
    __detail::__variant::__Traits<_Types...>::__S_move_assign,
    variant<_Types...>>
{
```

If the type so produced needs to be manually tagged as TR, one may need to also add TR tags to all the "detail" types used by the implementation, because the enforcement policy would forbid to mark as TR just the "end result".

It is unknown how vexing this is going to be in practice. The currently proposed model should have done extensive experimentation, and added the results to the proposal.

- There is a risk that **legacy code** will constitute a barrier for TR adoption. Since TR is a new language feature, no code written before C++26 is already using it. In particular no "Rule of Five" datatypes will be automatically TR in the proposed model.

This means that **a type will only be able to become TR when all of its "dependencies" will have been upgraded to C++26**. If one is creating a type `T` that uses a third-party legacy type `L` as data

member, then `T` will not be automatically be TR until `L`'s library is upgraded to C++26. If `T` is a Rule of Five type, it cannot be manually marked as TR, because doing so will make the program ill-formed:

```
struct T trivially_relocatable // possible ERROR here
{
    // third party types, outside our control:
    Lib1::Foo m_foo;
    Lib2::Bar m_bar;

    T();
    ~T();
};
```

One could use a conditional form:

```
struct T
    trivially_relocatable(
        std::is_trivially_relocatable_v<Lib1::Foo> &&
        std::is_trivially_relocatable_v<Lib2::Bar>
    ) // OK, how but tedious is this?
{
    Lib1::Foo m_foo;
    Lib2::Bar m_bar;

    T();
    ~T();
};
```

but this looks vexing. An "automatic opt-in" (invented syntax: `trivially_relocatable(auto)`) is not part of the currently adopted TR model.

As discussed in [§ 7.2 Leaving the TR status of Standard Library datatypes as QoI](#), **the Standard Library itself will be an offender**. In the currently adopted model, the TR status of library types is left to QoI, which effectively makes it a "legacy third-party" dependency for user code as far as TR is concerned.

- Finally, there is not a proposed way to have an "override mechanism" in order to manually mark a subobject as TR, in case we know that *it is* but the subobject type has not been marked as such by its author.

In conclusion: **we are unable to determine the practical impact of the enforcement model at scale, and therefore whether it constitutes a good trade-off.**

§ 9.2. Automatic TR is still not entirely safe

In the currently adopted model, TR somehow extends the "Rule of Five" to the "Rule of Six": a type that declares any of the special 5 member functions should define or delete them all, *and* it should have the `trivial_relocatable` keyword (possibly set to `false`).

Modern classes that adopts the "Rule of Zero" will naturally be TR if their subobjects are.

However, as noted in § 8.3 [Unclear behaviors for polymorphic classes](#), Rule of Zero types may be automatically TR and yet exhibit UB if they get trivially relocated. We are not sure if this problem can be addressed.

§ 10. P1144's trivial relocation model

[\[P1144R10\]](#) is an alternative proposal for trivial relocation. Its TR model differs from the currently adopted one in many (small) ways; please refer to [\[P2814R0\]](#) for a very detailed comparison.

A key summary of [\[P1144R10\]](#)'s mechanics for its TR model is:

- just like the currently adopted model, introduce "trivially relocatable" as a type property, for scalars, TR classes, arrays of TR types and cv-qualified versions;
- a class can be marked as TR (or not) by using an attribute (`[[trivially_relocatable]]`), with an optional boolean argument;
- a class is automatically TR if:
 - all of its subobjects are of TR type or of reference type; and
 - none of its eligible copy operations, move operations, or destructor are user-provided;
 - it has no virtual bases; and
 - no virtual member functions;
- there is no enforcement: a class can be manually marked as TR, even if some of the subobjects are not TR.

§ 10.1. Overall differences between P2786 and P1144

The "mechanical" differences between [\[P2786R4\]](#) and [\[P1144R10\]](#)'s TR models are summarized in this table:

		P2786	P1144
Type property		"Trivially relocatable": scalars, TR classes, arrays of TR, and cv-qualified TR	
TR tag		Contextual keyword: <code>trivially_relocatable(bool)</code>	Attribute: [[<code>trivially_relocatable(bool)</code>]]
A class is automatically TR if?	Subobjects	Must be of TR or reference type	
	Destructor	Not deleted, not user-provided	If eligible, not user-provided
	Copy/move ctor	Type must be move constructible via a non-deleted non-user-provided constructor	If eligible, not user-provided
	Copy/move assign	Irrelevant	If eligible, not user-provided
	Virtual bases	Not allowed	Not allowed
	Virtual member functions	Allowed	Not allowed
	TR tag	Must not be present	Irrelevant
TC¹ implies TR		No	Yes
Enforcement of TR tags		Yes	No

¹ Trivial Copyability

§ 10.2. P1144's TR semantics

Once more, we are not interested in comparing the exact shape of the APIs proposed, and we will try to focus instead on what kind design P1144 allows for.

P1144's requirements for TR types are *more strict* than the currently adopted proposal. (Or, vice versa: the currently adopted proposals "accepts" by default more types than P1144.) In particular, in P1144's model, types with user-provided assignment operations and types with virtual member functions are **not** automatically TR.

The reason for these additional restrictions is that P1144's TR semantics is also meant to cover assignments: in this model, a move assignment on a relocatable type is assumed to be equivalent to destruction of the target and move construction of the source. In other words, relocatable types have "value semantics" for assignments. This is important because it unlocks the [additional optimizations](#) that we have discussed earlier, optimizations that are unavailable in the currently adopted model.

This equivalence of move assignment with destruction and move construction is enshrined by the semantic requirements of the proposed [relocatable](#) concept; cf. §4.7 in [\[P1144R10\]](#).

If the user provides an assignment operator then we can no longer assume its semantics (it may have reference semantics), and therefore the type can no longer be automatically TR.

The presence of virtual functions also disables automatic TR, because in case the type gets sliced (by move construction or move assignment) then the virtual table pointer cannot be copied using a [memcpy](#).

§ 10.2.1. Trivially copyable always implies automatically trivially relocatable

In P1144's model the requirements for begin automatic TR are always satisfied by trivially copyable types. In contrast, in the currently adopted model, it is possible to have trivially copyable types that are *not* trivially relocatable.

A trivially copyable type may be missing some special operation; for instance, it may be not move constructible or not assignable:

```
struct TC {
    TC();
    TC(const TC &) = default;
    TC(TC &&) = default;
    TC &operator=(const TC &) = delete;
    TC &operator=(TC &&) = delete;
    ~TC() = default;
};

static_assert(    std::is_trivially_copyable_v<TC>);
static_assert(not std::is_move_assignable_v<TC>);
```

P1144 still considers those types to be TR. However, the APIs that make use of relocation may refuse to work with such types: [std::vector](#) still requires to [push_back](#) a non-movable type, and the proposed [std::relocate_at](#) building block effectively *Mandates* the type to be move constructible.

§ 10.2.2. TR is an intrinsic quality

If a type satisfies all the criteria for being automatically TR, then in P1144's model the type **is** TR, even if marked `[[trivially_relocatable(false)]]`. This makes it impossible to create a type which is trivially copyable, and yet not trivially relocatable.

We are not sure if P1144's intention is to have implementations issue QoI diagnostics for such types.

§ 10.2.3. Trivially copyable types are a subset of trivially relocatable

If we combine the last two considerations together, we can conclude that in P1144's model trivial copyable types are always a subset of trivial relocatable types. Of course the interesting cases are types that are *not* trivially copyable but only TR (e.g. "Rule Of Five" types like `std::unique_ptr<T>`); these are supposed to be manually marked. This means that the subset is also proper.

This result has a practical application: generic that wants to apply TR-related optimizations can use TR to also handle trivially copyable types.

§ 10.3. Which optimizations are possible in P1144's model?

Since P1144 is more strict than the currently adopted proposal, it allows for more optimization opportunities. In particular, it unlocks both family of optimizations discussed above:

- vector reallocation can get optimized, just like in the adopted model, as discussed in [§ 5 What optimizations are possible in the current model?](#);
- optimizations for move assignments are also enabled: vector `erase` and `insert`; `swap`; and swap-based algorithms (as discussed in [§ 6 The missing optimizations](#)).

§ 10.4. Which optimizations are **not possible** in P1144's model?

Types for which move assignment is not equivalent to destruction of the target and move construction from the source are not considered to be TR in P1144's model. As discussed in [§ 6.1 Why are TR optimizations for move assignments not allowed?](#), examples of these types include `IRef`, `std::tuple<int &>`, as well as types such as `std::pmr::string`.

This means that in P1144's model a vector of `std::pmr::string` will not be able to exploit TR when it reallocates.



This is the "split" that we were referring to in the § 2 Introduction chapter: this perfectly reasonable optimization is possible only in the currently adopted model, but not in P1144's; while instead optimizing insertion, erasure, sorting, etc. of a `std::vector<std::unique_ptr<T>>` is only possible in P1144 but not in the currently adopted model.

There does not seem to be any technical reason as of why *both* optimizations shouldn't be available instead; the only obstacle seems to be on how to "offer" these facilities (and, thus, concerns relative to naming, teachability, discoverability).

§ 11. Summary

We have identified two different trivial relocation models, that enable very different optimization possibilities:

Model	Optimizes	Enabled by
1. Move construction and destruction of the source object can be achieved via <code>memcpy</code> for a TR type	Vector reallocation	Both P2786 and P1144
2. Like 1., <i>and</i> , TR types have value semantics: move assignment followed by destruction of the source is equivalent to destruction of the target and relocation	Vector reallocation, <code>insert</code> , <code>erase</code> ; <code>swap</code> ; swap-based algorithms (e.g. <code>sort</code> , <code>rotate</code>)	P1144 only (<i>unacceptable library solution provided by P2959</i>)

The currently adopted model only allows for n° 1.



This table summarizes the support for non-trivially copyable types on popular implementations:

Type	TR for move construction only?	TR for move construction and assignment? ¹	Notes
<code>std::string</code>	🟡	🟡	On some implementations it is self-referential.
<code>std::shared_ptr<T></code> , <code>std::weak_ptr<T></code>	✅	✅	

Type	TR for move construction only?	TR for move construction and assignment? ¹	Notes
<code>std::optional<T></code> , <code>std::variant<T></code>	✓	✓	Provided that <code>T</code> is a type with the same TR capabilities
<code>std::vector<T></code>	✓	✓	
<code>std::list<T></code>	●	●	On some implementations it is self-referential.
<code>std::tuple<T &></code>	✓	✗	The assignment operator implements reference semantics.
<code>std::pmr::string</code>	✓	✗	The allocator is not assignable.
Polymorphic types	✓ (???)	✗	Slicing may introduce UB . However, normally polymorphic types are not movable nor assignable.

¹ Note: the second column always implies (generalizes) the first.

§ 12. A possible way forward

§ 12.1. Disclaimer

We would like to ask forgiveness in advance for our sin: it is very presumptuous of us to propose ideas, but then ask others to actually do the work to support these ideas.

As we mentioned in the [§ 2 Introduction](#) chapter, we are only sketching a possible plan to reconcile the two TR proposals. The plan that we put forward is very bold, and we certainly understand if readers are skeptical about its feasibility.

§ 12.2. Proposed action points

- We strongly urge EWG and LEWG to reconsider the adoption of [\[P2786R4\]](#) as the model for trivial relocation in C++.
- Ideally, the two TR proposals should be merged into one.

- If this is not possible or not wanted, then we would prefer [\[P1144R10\]](#) for C++26 instead of [\[P2786R4\]](#).
- **The merged proposal should give proper name and semantics to:**
 1. the combined "move construction + destruction of the source" operation (relocation? destroying move?); and
 2. the type property that describes that, for a given type, "move assignment + destruction of the source" is equivalent to "destruction of the target + move construction + destruction of the source" (that is, is equivalent to "destruction of the target + relocation").
- **The merged proposal should have two language enablers:**
 - one that enables trivial destroying-move and only that; and
 - one that enables it, and says that a type has the type property described above.
- **The latter enabler should have the simpler, more generic name;** most TR types will want to use that enabler.
 - This is important for teachability and discoverability of the feature.
 - We believe that "trivially copyable" is the precedent to follow here, in the sense that it's the type trait that is actually useful and has a very simple name. Its building blocks, like "trivially copy constructible", have longer/uglier names.
- Since we are proposing *two* language enablers, **an attribute sounds more appetizing than two keywords**. However, if enforcement semantics are wanted, the current stance on attribute ignorability requires the proposal to add keywords.
- Given that we are adding a tool that removes "Undefined Behavior That Works In Practice", but adds another source of UB, **we would like to have SG12's expressed vote on whether TR should embrace enforcement or not**.
 - The pros and cons of having an enforcement model should be clearly analyzed before choosing which direction to follow; we offered some starting points for discussion in the [§ 9 Enforcement model](#) chapter.
- **Language and library must be dealt with in the very same proposal.** At a minimum, the library utilities with the biggest impact should be included.
 - That includes for instance `std::uninitialized_relocate`, as it's the API that everyone will want to use.
 - If enforcement is wanted, then the proposal **must** also include *all* of the Standard Library datatypes that won't be automatically TR and thus need manual marking. TR should not ship in C++26 unless the Standard Library is also ready for it. Leaving the TR status of the Standard Library to QoI makes for a poorly cooked feature.

§ 13. Acknowledgements

Thanks to KDAB for supporting this work.

All remaining errors are ours and ours only.

§ References

§ Informative References

[P1144R10]

Arthur O'Dwyer. *[std::is_trivially_relocatable](https://wg21.link/p1144r10)*. 15 February 2024. URL: <https://wg21.link/p1144r10>

[P2786R4]

Mungo Gill, Alisdair Meredith. *[Trivial Relocatability For C++26](https://wg21.link/p2786r4)*. 9 February 2024. URL: <https://wg21.link/p2786r4>

[P2814R0]

Mungo Gill, Alisdair Meredith; Arthur O'Dwyer. *[Trivial Relocatability --- Comparing P1144 with P2786](https://wg21.link/p2814r0)*. 19 May 2023. URL: <https://wg21.link/p2814r0>

[P2959R0]

Alisdair Meredith. *[Container Relocation](https://wg21.link/p2959r0)*. 15 October 2023. URL: <https://wg21.link/p2959r0>

[P2967R0]

Alisdair Meredith. *[Relocation Is A Library Interface](https://wg21.link/p2967r0)*. 15 October 2023. URL: <https://wg21.link/p2967r0>

[P2996R2]

Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. *[Reflection for C++26](https://wg21.link/p2996r2)*. 15 February 2024. URL: <https://wg21.link/p2996r2>